WEEK-5
Collective Communications in MPI

Niharika Gawade
240905284
Roll no:30
CSE-D

1. 1) Implement a MPI program to read N values in the root process. Root process sends one value to each process. Every process receives it and finds the factorial of that number and returns it to the root process. Root process gathers the factorial and finds sum of it. Use N number of processes.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
int rank, n, x, fact = 1, sum = 0;
int ans[100];

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &n);

if (rank == 0) {
int a[100];

printf("Enter %d values: ", n);
for (int i = 0; i < n; i++)
scanf("%d", &a[i]);

MPI_Scatter(a, 1, MPI_INT,
&x, 1, MPI_INT,
0, MPI_COMM_WORLD);
}
else {
MPI_Scatter(NULL, 1, MPI_INT,
&x, 1, MPI_INT,
0, MPI_COMM_WORLD);
}

for (int i = 1; i <= x; i++)
fact *= i;

MPI_Gather(&fact, 1, MPI_INT,
ans, 1, MPI_INT,
0, MPI_COMM_WORLD);

if (rank == 0) {
```

```
for (int i = 0; i < n; i++)
sum += ans[i];

printf("Sum of factorials = %d\n", sum);
}

MPI_Finalize();
return 0;
}
```

OUTPUT



2) Implement a MPI program to read an integer value M and NXM elements into an ID array in the root process, where N is the number of processes. Root process sends M elements to each process. Each process finds average of M elements it received and sends these average values to root. Root collects all the values and finds the total average. Use collective comunication routines.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
int rank, n, M;
float local[100], data[100], avgs[100];
float avg = 0, total = 0;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &n);


if (rank == 0) {
printf("Enter M: ");
scanf("%d", &M);

printf("Enter %d elements:\n", n * M);
for (int i = 0; i < n * M; i++)
scanf("%f", &data[i]);
}


MPI_Bcast(&M, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```
MPI_Scatter(data, M, MPI_FLOAT,
local, M, MPI_FLOAT,
0, MPI_COMM_WORLD);


for (int i = 0; i < M; i++)
avg += local[i];

avg = avg / M;


MPI_Gather(&avg, 1, MPI_FLOAT,
avgs, 1, MPI_FLOAT,
0, MPI_COMM_WORLD);


if (rank == 0) {
for (int i = 0; i < n; i++)
total += avgs[i];

total = total / n;

printf("Overall Average = %.2f\n", total);
}

MPI_Finalize();
return 0;
}
```

OUTPUT



3) Implement a MPI program to read a string. Using N processes (string length is evenly divisible by N), find the number of non-vowels in the string. In the root process print number of non-vowels found by each process and print the total number of non-vowels.

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc,char *argv[]) {
int rank,n,len,local,total;
char str[100],part[100];

MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&n);

if(rank==0) {
printf("Enter string: ");
scanf("%s",str);
len=strlen(str);
}

MPI_Bcast(&len,1,MPI_INT,0,MPI_COMM_WORLD);

MPI_Scatter(str,len/n,MPI_CHAR,
part,len/n,MPI_CHAR,0,MPI_COMM_WORLD);

local=0;
for(int i=0;i<len/n;i++)
if(!strchr("aeiouAEIOU",part[i]))
local++;

MPI_Reduce(&local,&total,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);

if(rank==0)
printf("Total non-vowels = %d\n",total);

MPI_Finalize();
return 0;
}
```

OUTPUT

```
Enter string: hello
Total non-vowels = 3
```

4) Implement a MPI Program to read two strings S1 and S2 of same length in the root process. Using N processes including the root (string length is evenly divisible by N), produce the resultant string as shown below. Display the resultant string in the root process. Use Collective communication routines.
Example:
String S1: string String S2: length

Resultant String: slternigntgh

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
int rank, n, len;
char s1[100], s2[100];
char local1[100], local2[100];
char result[200], local[200];

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &n);

if (rank == 0) {
printf("Enter S1: ");
scanf("%s", s1);

printf("Enter S2: ");
scanf("%s", s2);

len = strlen(s1);
}

MPI_Bcast(&len, 1, MPI_INT, 0, MPI_COMM_WORLD);


MPI_Scatter(s1, len / n, MPI_CHAR,
local1, len / n, MPI_CHAR,
0, MPI_COMM_WORLD);

MPI_Scatter(s2, len / n, MPI_CHAR,
local2, len / n, MPI_CHAR,
0, MPI_COMM_WORLD);


for (int i = 0; i < len / n; i++) {
local[2 * i] = local1[i];
local[2 * i + 1] = local2[i];
}


MPI_Gather(local, 2 * (len / n), MPI_CHAR,
result, 2 * (len / n), MPI_CHAR,
0, MPI_COMM_WORLD);

if (rank == 0) {
result[2 * len] = '\0';
```

```
printf("Resultant string = %s\n", result);
}

MPI_Finalize();
return 0;
}
```

OUTPUT

```
Enter S1: string
Enter S2: length
Resultant string = slternigntgh
```

5) Implement parallel selection sort [hint: Use merge sort after sorting subarrays using parallel selection sort]

```
#include <mpi.h>
#include <stdio.h>

void selection(int a[],int n) {
for(int i=0;i<n-1;i++) {
int k=i;
for(int j=i+1;j<n;j++)
if(a[j]<a[k]) k=j;

int t=a[i];
a[i]=a[k];
a[k]=t;
}
}

void merge(int a[],int n) {
for(int i=1;i<n;i++) {
int x=a[i],j=i-1;
while(j>=0 && a[j]>x) {
a[j+1]=a[j];
j--;
}
a[j+1]=x;
}
}

int main(int argc,char *argv[]) {
int rank,p,n;
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&p);

if(rank==0) {
printf("Enter array size (multiple of %d): ",p);
```

```
scanf("%d",&n);
}

MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);

int a[n],local[n/p];

if(rank==0) {
printf("Enter array: ");
for(int i=0;i<n;i++) scanf("%d",&a[i]);
}

MPI_Scatter(a,n/p,MPI_INT,
local,n/p,MPI_INT,0,MPI_COMM_WORLD);

selection(local,n/p);

MPI_Gather(local,n/p,MPI_INT,
a,n/p,MPI_INT,0,MPI_COMM_WORLD);

if(rank==0) {
merge(a,n);

printf("Sorted array: ");
for(int i=0;i<n;i++)
printf("%d ",a[i]);
printf("\n");
}

MPI_Finalize();
return 0;
}
```

OUTPUT

```
Enter array size (multiple of 4): 8
Enter array: 2
1
4
5
6
8
3
7
Sorted array: 1 2 3 4 5 6 7 8
```